NAME	PYNAMUDI JHANSY
REGISTER NUMBER	99230040740
DEPARTMENT	CSE

MINI ASSESSMENT

API Handling, JSON Parsing and Error Management

1. If an API returns 429, what should your program do?

Answer: A **429 Too Many Requests** response means the client has sent too many requests in a given period. The program should not continuously retry immediately. Instead, it should wait for some time and then retry the request. If the server provides a **Retry-After** header, the program should follow that value. Otherwise, an **exponential backoff** strategy can be used, where the waiting time increases after each failed attempt. The program should also set a reasonable maximum number of retries. This prevents the application from creating unnecessary traffic and helps protect the API from further rate limiting. Logging the event can also help developers monitor API usage.

2. If response.json() fails, what should your program do?

Answer: response.json() can fail when the server response is not valid JSON. For example, the server might return HTML, plain text, an empty response, or an unexpected error page. The program should therefore **not assume that every successful HTTP request contains valid JSON**. The JSON parsing operation should be placed inside appropriate exception handling. If parsing fails, the program can record the error, inspect the response status and content type, and return a safe error message or fallback value. This prevents the application from crashing unexpectedly. It is also useful to check the HTTP status code before attempting to process the response. This makes the program more reliable when working with external APIs.

3. If a JSON key is missing, why might .get() be safer than []?

Answer: When accessing a dictionary, using **data["key"]** expects the key to exist. If the key is missing, Python raises a **KeyError**, which may stop the program unless the error is handled. The **.get()** method is safer when a key may be absent. For example, data.get("name") returns **None** if the key does not exist. A default value can also be supplied, such as data.get("name", "Unknown"). Therefore, .get() is useful for optional JSON fields and for APIs whose responses may vary. Direct [] access is appropriate when the key is required and its absence should be treated as an error.

Additional Important Concepts

Rate limiting: APIs often limit how many requests a client can make in a specific time period. Rate limiting protects server resources and maintains fair access for users.

Retry with backoff: Instead of retrying instantly, the program waits progressively longer between attempts. This reduces pressure on the server.

Error handling: External API calls can fail because of network problems, server errors, invalid responses, or rate limits. Robust programs handle these cases gracefully.

JSON validation: Before using data returned by an API, the program should verify that the response has the expected structure and required fields.

Default values: Using .get() with a default value allows the program to continue safely when optional information is not present.

Example Python Code

```
import requests
import time

response = requests.get("https://example.com/api")

if response.status_code == 429:
print("Too many requests. Retrying after a delay...")
time.sleep(2)

try:
data = response.json()
name = data.get("name", "Unknown")
print(name)
except ValueError:
print("Response is not valid JSON.")
```

Conclusion: Reliable API programs should handle rate limits, parsing errors, missing JSON fields, network failures, and unexpected server responses. Using retries with backoff, exception handling, validation, and safe dictionary access makes an application more stable and user-friendly.